

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	B.Nandini	Reg. No.:	192525137
Date:	19-08-2026	Duration:	60 minutes

Code of Conduct

I B. Nandini (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B. Nandini

Annexure A – Code Review & Repository Evaluation

Agriculture Equipment Rental and Farm Resource Management Platform

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.

Problem Statement

Farmers, especially small and medium-scale farmers, often cannot afford to purchase expensive agricultural equipment such as tractors, harvesters, ploughing machines, seeders, sprayers, and irrigation equipment. At the same time, equipment owners may have machines that remain unused for significant periods.

The **Agriculture Equipment Rental and Farm Resource Management Platform** provides a digital platform that connects farmers with agricultural equipment owners. Farmers can search for equipment, check availability, view rental prices, submit rental requests, and manage their bookings. Equipment owners can add equipment, manage availability, approve rental requests, and track rental activities.

Implemented Solution

The proposed system is a full-stack web application consisting of:

- Farmer interface
- Equipment-owner interface
- Admin interface
- Equipment management
- Equipment search
- Rental booking
- Availability tracking
- User authentication
- Payment management
- Rental history
- Farm-resource management
- Reports and notifications

- Describe the project objectives and key functionalities.

Project Objectives

- 1. Provide easy access to agricultural equipment.
- 2. Reduce the cost of purchasing expensive machinery.
- 3. Improve agricultural equipment utilization.
- 4. Connect farmers and equipment owners.
- 5. Simplify equipment rental booking.
- 6. Maintain equipment availability information.
- 7. Manage rental records digitally.
- 8. Provide secure user authentication.
- 9. Maintain farmer and equipment-owner information.
- 10. Improve farm resource management.

Key Functionalities

Functionality	Description
User Registration	Allows farmers and owners to create accounts
Login	Provides secure user authentication
Equipment Listing	Owners can add agricultural equipment
Equipment Search	Farmers can search available equipment
Equipment Filtering	Filter by type, location and price
Booking	Farmers can request equipment
Approval	Owners can accept or reject requests
Availability	Tracks equipment availability
Payment	Maintains rental payment information
Rental History	Stores previous rental activities
Resource Management	Tracks farm equipment/resource usage
Admin Management	Admin manages users and equipment

Expected Outcome

The system provides a convenient and centralized platform for agricultural equipment rental. It reduces equipment acquisition costs, improves machinery utilization, simplifies rental management, and supports better farm-resource planning.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.

agriculture-equipment-platform/

|

```
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── assets/
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── Dockerfile
|
├── backend/
│   ├── controllers/
│   ├── models/
│   ├── routes/
│   ├── middleware/
│   ├── services/
│   ├── config/
│   ├── app.js
│   ├── package.json
│   └── Dockerfile
|
├── database/
│   └── schema.sql
```

```
|   └── seed.sql
|
|
├── tests/
|   ├── frontend/
|   └── backend/
|
|
├── docs/
|   ├── architecture.png
|   ├── workflow.png
|   └── report.pdf
|
|
├── .gitignore
├── .env.example
├── docker-compose.yml
├── README.md
└── LICENSE
```

- Justify how the repository layout supports maintainability and collaboration.

Evaluation of Repository Structure

The repository is logically divided into frontend, backend, database, testing, and documentation components.

Folder Purposes

frontend/

Contains the user interface of the application.

backend/

Contains server-side logic and APIs.

database/

Contains database schema and sample data.

tests/

Contains test cases for frontend and backend.

docs/

Contains architecture diagrams, workflow diagrams, and project documentation.

.gitignore

Prevents unnecessary files such as `node_modules` and `.env` from being committed.

.env.example

Shows the environment variables required by the project without exposing confidential values.

docker-compose.yml

Defines the containers required to execute the complete application.

README.md

Provides project information, installation instructions, commands, and usage information.

Maintainability

The repository supports maintainability because each component has a specific responsibility. Developers can modify frontend, backend, database, or testing components independently.

Collaboration

The organized structure allows multiple developers to work on separate modules simultaneously and reduces confusion during collaborative development.

File Naming Conventions

Recommended naming:

```
EquipmentController.js
EquipmentModel.js
equipmentRoutes.js
BookingController.js
BookingModel.js
UserController.js
```

Consistent naming makes files easier to locate and understand.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.

The assignment asks for review of **readability, modularity, coding standards, documentation, strengths, and areas for improvement.**

Readability

The code should use:

- Meaningful variable names.
- Consistent indentation.
- Proper spacing.
- Small functions.
- Comments where required.
- Clear API names.

Example:

```
const availableEquipment = await Equipment.find({
  availability: "Available"
});
```

This is easier to understand than:

```
const x = await E.find({a:"A"});
```

Modularity

The application is divided into independent modules:

```
Authentication
Equipment Management
Booking Management
Payment Management
User Management
Resource Management
```

This makes the application easier to modify and test.

Coding Standards

Recommended standards:

- Use camelCase for JavaScript variables and functions.
- Use meaningful class/component names.
- Avoid duplicate code.
- Use consistent indentation.
- Use reusable components.
- Validate user input.
- Handle exceptions properly.

Example of Modular Code

```
const createEquipment = async (req, res) => {
  try {
    const equipment = await Equipment.create(req.body);
```

```
res.status(201).json({  
  message: "Equipment added successfully",  
  equipment  
});  
  
} catch (error) {  
  
  res.status(500).json({  
    message: "Unable to add equipment"  
  });  
  
}  
  
};
```

- Identify strengths and areas for improvement.

Strengths

- Modular project structure.
- Clear separation between frontend and backend.
- Reusable components.
- Meaningful variable names.
- REST API-based communication.
- Error handling.
- Database separation.
- Docker support.
- Git-based version control.

Areas for Improvement

- Add more automated tests.
- Improve API documentation.
- Add stronger input validation.
- Improve error messages.
- Add centralized error handling.
- Improve security.
- Add code formatting tools.
- Add continuous integration.
- Improve logging and monitoring.

Code Quality Evaluation

Overall, the proposed code organization provides **good readability, modularity, maintainability, and scalability**, but automated testing, documentation, validation, and security can be further improved.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.

The assignment requires evaluation of **commit history, branching strategy, commit messages, and repository maintenance practices**.

Git Branching Strategy

A Git Flow-based approach can be used:

```
main
|
develop
|
+-- feature/authentication
|
+-- feature/equipment-management
|
+-- feature/rental-booking
|
+-- feature/payment
|
+-- feature/resource-management
|
release/v1.0
|
hotfix/booking-error
```

Main Branch

Contains stable production-ready code.

Develop Branch

Contains integrated development work.

Feature Branches

Used for individual features.

Example:

```
git checkout -b feature/equipment-management
```

Release Branch

Used for final testing before deployment.

Hotfix Branch

Used for critical production problems.

Sample Commit History

```
Initial project structure
Add farmer authentication
Add equipment owner module
Implement equipment listing
Add equipment search
Implement equipment booking
Add equipment availability
Add payment module
Add farm resource management
Add Docker configuration
Fix booking validation
Update project documentation
```

Good Commit Messages

Good:

```
Add equipment availability management
```

Poor:

```
changes
```

Good:

```
Fix rental booking validation
```

Poor:

```
final update
```

Git Commands

```
git status
git add .
git commit -m "Add equipment management module"
git branch
git checkout develop
git merge feature/equipment-management
git pull
git push
git log --oneline
```

- Explain how version control supports collaborative software development.

How Git Supports Collaboration

Git:

- Allows multiple developers to work simultaneously.
- Maintains project history.
- Provides feature branches.
- Supports code review.
- Allows merging.
- Helps resolve conflicts.
- Allows previous versions to be restored.
- Provides a centralized remote repository.

Version Control Evaluation

The project can achieve good version-control practices by using meaningful commits, feature branches, regular synchronization, pull requests, and proper repository maintenance.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.

Test ID	Test Case	Input	Expected Result
TC01	Farmer Registration	Valid user details	Account created
TC02	Farmer Login	Valid username/password	Login successful
TC03	Invalid Login	Incorrect password	Error message displayed
TC04	Add Equipment	Valid equipment details	Equipment added
TC05	Search Equipment	Tractor	Tractor results displayed
TC06	Filter Equipment	Location + price	Matching equipment displayed
TC07	Book Equipment	Available equipment	Booking request created
TC08	Book Unavailable Equipment	Unavailable equipment	Booking rejected
TC09	Owner Approval	Valid booking	Booking approved
TC10	Owner Rejection	Booking request	Booking rejected
TC11	Cancel Booking	Existing booking	Booking cancelled
TC12	Equipment Return	Completed rental	Status changed to returned
TC13	Payment	Valid payment details	Payment recorded
TC14	Rental History	Logged-in farmer	Previous rentals displayed

Test ID	Test Case	Input	Expected Result
TC15	Logout	Logged-in user	User logged out

- Present test cases, execution results, and screenshots as evidence.

Example Execution Results

TC01 - Farmer Registration	PASS
TC02 - Farmer Login	PASS
TC03 - Invalid Login	PASS
TC04 - Add Equipment	PASS
TC05 - Search Equipment	PASS
TC06 - Filter Equipment	PASS
TC07 - Book Equipment	PASS
TC08 - Unavailable Equipment	PASS
TC09 - Owner Approval	PASS
TC10 - Owner Rejection	PASS
TC11 - Cancel Booking	PASS
TC12 - Equipment Return	PASS
TC13 - Payment	PASS
TC14 - Rental History	PASS
TC15 - Logout	PASS

Docker Testing

Start the application:

```
docker compose up -d
```

Check containers:

```
docker ps
```

Expected:

```
frontend    Up
backend     Up
database    Up
```

Check logs:

```
docker compose logs
```

Validation

The application is considered successfully validated when:

- Users can register.
- Users can log in.

- Equipment can be added.
- Equipment can be searched.
- Available equipment can be booked.
- Owners can approve requests.
- Rental records are stored.
- Containers run without errors.
- Database communication works correctly.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

The assignment requires evaluation of the **README, installation instructions, usage guide, and dependencies**.

README Structure

The README.md should contain:

```
# Agriculture Equipment Rental and Farm Resource Management Platform

## Project Description

## Objectives

## Features

## Technologies Used

## System Requirements

## Installation

## Project Structure

## Environment Variables

## Running the Application

## Docker Setup

## Testing

## Git Workflow

## API Documentation

## Future Enhancements

## Contributors
```

Installation Instructions

Example:

```
git clone https://github.com/username/agriculture-equipment-platform.git  
cd agriculture-equipment-platform
```

Install frontend dependencies:

```
cd frontend  
npm install
```

Install backend dependencies:

```
cd ../backend  
npm install
```

Docker Installation

```
docker compose build  
docker compose up -d
```

Check Containers

```
docker ps
```

Stop Application

```
docker compose down
```

Dependencies

Frontend:

```
React  
React Router  
Axios
```

Backend:

```
Node.js  
Express  
Database Driver  
JWT  
bcrypt
```

Development:

```
Git  
Docker  
Docker Compose  
VS Code
```

- Suggest improvements to enhance usability and maintainability.

Documentation Strengths

- Project purpose is clearly defined.
- Installation steps can be provided.
- Dependencies are listed.
- Docker commands are documented.
- Repository structure is explained.

Documentation Improvements

1. Add screenshots.
2. Add API documentation.
3. Add troubleshooting instructions.
4. Add database setup instructions.
5. Add sample environment variables.
6. Add contribution guidelines.
7. Add testing instructions.
8. Add deployment instructions.
9. Add architecture diagrams.
10. Add version information.

7. Code Review Findings and Recommendations

- Summarize the overall code review findings.

Overall Findings

The Agriculture Equipment Rental and Farm Resource Management Platform has a modular full-stack architecture. The separation of frontend, backend, database, and testing components improves maintainability.

The repository structure supports collaborative development, while Git provides version tracking and branch management. Docker improves portability and provides a consistent execution environment.

Code Quality Findings

Strengths:

- Modular architecture.
- Separation of concerns.
- Reusable components.
- Clear naming.
- API-based communication.
- Error handling.

- Structured project organization.

Weaknesses:

- More automated testing is required.
- API documentation can be improved.
- Security validation should be strengthened.
- Logging can be improved.
- More comments/documentation can be added where necessary.

Repository Findings

Strengths:

- Logical folder structure.
- Separate frontend and backend.
- Git version control.
- Docker configuration.
- Documentation folder.

- Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Improvements:

- Add branch protection.
- Use pull requests.
- Add issue tracking.
- Add contribution guidelines.
- Use automated CI/CD.

Testing Findings

Testing should cover:

- Authentication.
- Equipment management.
- Booking.
- Availability.
- Payment.
- User permissions.
- Database operations.
- API responses.

Automated unit and integration testing should be added to improve reliability.

Documentation Findings

The project should maintain:

- README.
- Installation guide.
- API documentation.
- Database documentation.
- Docker instructions.
- Testing instructions.
- Architecture diagrams.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

